ASSIGNMENT 0 : PYTHON

1. NIFTY 50 Dataset Analysis and Basic Visualization

Dataset Instructions:- You are required to fetch the NIFTY 50 dataset yourself using Python. Use the **yfinance** library to download daily data for the past one month.

- **Ticker:** "^NSEI"
- **Library**: yfinance
- **Duration:** Past 1 month (choose appropriate start and end dates)

You may use the following code snippet to fetch the data:

```
import yfinance as yf
df = yf.download(ticker, start=start_date, end=end_date)
```

After downloading the data perform the following operations:

1.1 Store the downloaded data in a pandas DataFrame, and print the shape of the dataset, the column names, and the first 5 rows.

1.2 Since yfinance automatically provides the dates as the dataframe index, print the index and confirm that it is a Datetime Index.If the index is not in datetime format, convert it using `pd.to_datetime()`

1.3 using the dataframe: find maximum value in High, minimum value in Low and add a new column called Range= high-low .

1.4 plot the Close price vs time with title and labels.

1.5 Make a bar plot of the first 10 days using the Date index on the x-axis and Range on the y-axis.

1.6 Create a scatter plot of **Volume** vs **Close** to observe whether trading volume is related to price movement. Add appropriate axis labels and a title.

1.7 Add a new column to your DataFrame that shows how the closing price changes from one day to the next (in terms of percentage or relative change).After creating this column, print a summary of its basic statistics such as the average, minimum, and maximum values.

1.8 Add two new columns to your DataFrame that help you smooth the closing price:

1. MA5 — the 5-day moving average of the closing price
   (This means the average of the closing prices of the most recent 5 days.)
2. MA10 — the 10-day moving average of the closing price
   (This means the average of the closing prices of the most recent 10 days.)

After creating these two columns:

- Print the first few values of MA5 and MA10.
- Plot the closing price, MA5, and MA10 together on the same graph to compare how they move.

```
df['MA5'] = df['Close'].rolling(window=5).mean()
```

This is the code to compute MA5. Similarly you can compute MA10.

1.9 Rename the following columns:-(do this in last step)

"Volume" → "traded_volume"
"Close" → "adj_close"

2. Synthetic Data Generation and Visualization

A student named Asha is practicing basic data generation and plotting using Python. She wants to create a simple synthetic dataset and visualize it. Your task is to help her by performing the following operations:-

Write a program to:

1. Take N (number of data points) as input from the user.
2. Generate N random values for X uniformly between any range (for example 0 to 10).
3. Create Y using a simple mathematical relationship such as: Y=2X+3 or Y=X^2+5

(You can choose any basic function: linear, square, cube, or sine.)

Create the following plots and add proper titles and labels for each plot.

1. Scatter plot of X vs Y
2. Line plot of X vs Y (sorted by X)
3. Histogram of X

To ensure reproducibility, set a random seed (for example, 10 or 42) before generating the data.

A random seed is a number that we set so that the random values generated by the computer remain the same every time the program is run.This makes our results reproducible.

Functions to Implement: To organize your code, implement these functions:

- generate_data(N) → returns X and Y
- plot_scatter(X, Y)
- plot_line(X, Y)
- plot_histogram(X)
- set_seed(seed)

3. Rock-Paper-Scissors

RPS is a popular hand game played between two players. The rules of the game are as follows:

• Rock beats Scissors.

• Scissors beats Paper.

• Paper beats Rock.

Imagine you and your friend want to decide who gets the last slice of pizza. Instead of arguing, you both agree to play a fair Rock-Paper-Scissors game. The winner gets the pizza, and if it's a tie, you play again.

Write a Python program that simulates a Rock-Paper-Scissors game between a human player and the computer. Your program should:

1. Allow the user to input their choice (Rock, Paper, or Scissors).

2. Generate a random choice for the computer.
3. Determine the winner using the rules of the game.
4. Display the choices of both the user and the computer.
5. Allow the user to play multiple rounds until they decide to quit. [By inserting the ’Quit’ command (case independent)].

Constraints:

• The user’s input should be case-insensitive (e.g., ”rock”, ”Rock”, or ”ROCK” should all be valid).
• The program should handle invalid inputs and prompt the user to enter a correct choice.
• The program should display the total score (wins, losses, and ties) after each round.
• Use the random module to generate the computer’s choice.

4. Temperature Analysis Task

A weather researcher has collected temperature readings from a Himalayan station for m days. You are given these temperatures in a list. Your task is to help the researcher understand how the temperature changed over these days.

Note: You may create your own list of temperature values (for example, 5–10 numbers) or take the values as input from the user. No external dataset is required.

Write a Python function that takes this list of temperatures and computes the following:

1. Average temperature (mean)
2. Middle temperature (median)
3. Standard deviation – this tells how much the temperatures vary from the average
4. Variance – use the formula for sample variance, where the division is done by *(m−1)*

These values will help the researcher understand how stable or unstable the temperatures were.

Functions to implement: `analyze temperatures(temperatures)`

This function should:

- Print the mean, median, standard deviation, and variance
- Use Python’s built-in statistical library (like the statistics module)
- Handle special cases like: When the list is empty and when the list has only one temperature

This will ensure that your code does not crash due to division errors.